```
            void unlock()        <-- 3
            {
                lk.unlock();
                self->set_clear_mutex.unlock();
            }

            void lock()
            {
                std::lock(self->set_clear_mutex,lk);     <-- 4
            }

            ~custom_lock()
            {
                self->thread_cond_any=0;          <-- 5
                self->set_clear_mutex.unlock();
            }
        };
        custom_lock cl(this,cv,lk);
        interruption_point();
        cv.wait(cl);
        interruption_point();
    }
        // rest as before
};

template<typename Lockable>
void interruptible_wait(std::condition_variable_any& cv,
                        Lockable& lk)
{
    this_thread_interrupt_flag.wait(cv,lk);
}
```

Your custom lock type acquires the lock on the internal `set_clear_mutex` when it's constructed ❶ and then sets the `thread_cond_any` pointer to refer to the `std::condition_variable_any` passed in to the constructor ❷. The `Lockable` reference is stored for later; this must already be locked. You can now check for an interruption without worrying about races. If the interrupt flag is set at this point, it was set before you acquired the lock on `set_clear_mutex`. When the condition variable calls your `unlock()` function inside `wait()`, you unlock the `Lockable` object *and the internal* `set_clear_mutex` ❸. This allows threads that are trying to interrupt you to acquire the lock on `set_clear_mutex` and check the